GUÍA TÉCNICA DE DESARROLLO GRÁFICO CON PROCESSING

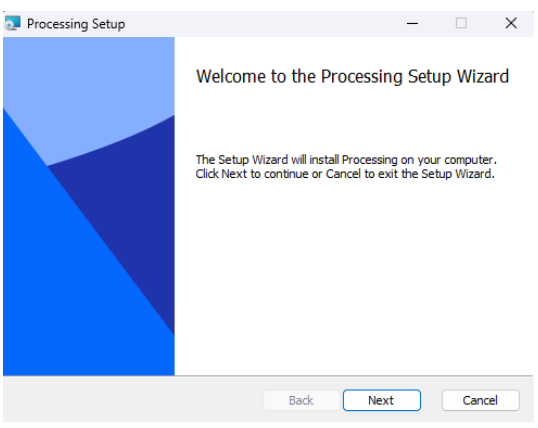
En esta guía se muestra de manera práctica el uso de Processing para desarrollar gráficos mediante programación. Se explican los pasos básicos para instalar y configurar la herramienta, crear un lienzo, utilizar primitivas geométricas y modificar sus propiedades visuales.

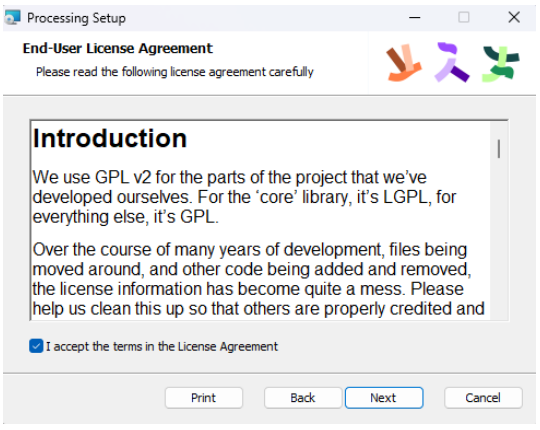
Como práctica se desarrollará un robot utilizando diferentes figuras geométricas. Al final se realiza una comparación entre ambas herramientas para identificar algunas diferencias en su sintaxis y forma de trabajo.

1. Instalación y configuración de Processing

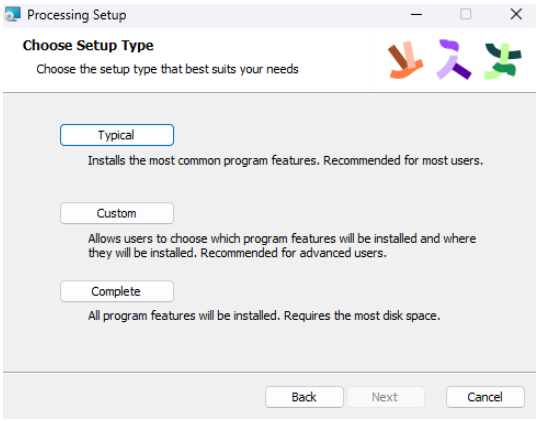
   Para comenzar se debe descargar Processing desde su sitio oficial e instalar la versión correspondiente al sistema operativo: https://processing.org/download?utm_source

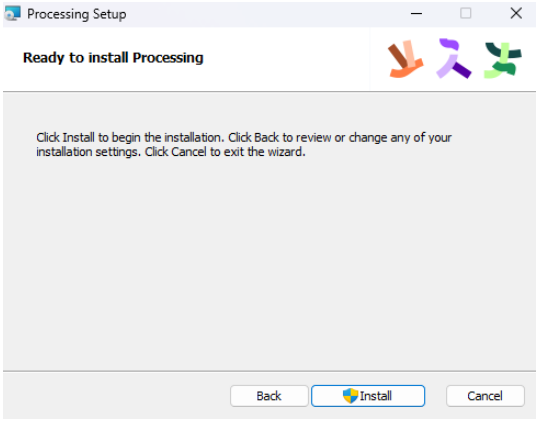
   Una vez instalado, se ejecuta el instalador y seleccionamos next. Posteriormente aceptamos los términos y condiciones, luego seleccionamos next:





Por último seleccionamos la opción Typical y damos a next. Una vez hecho eso solo seleccionamos Install y una vez terminada la instalación seleccionamos la última opción que dice Finish:

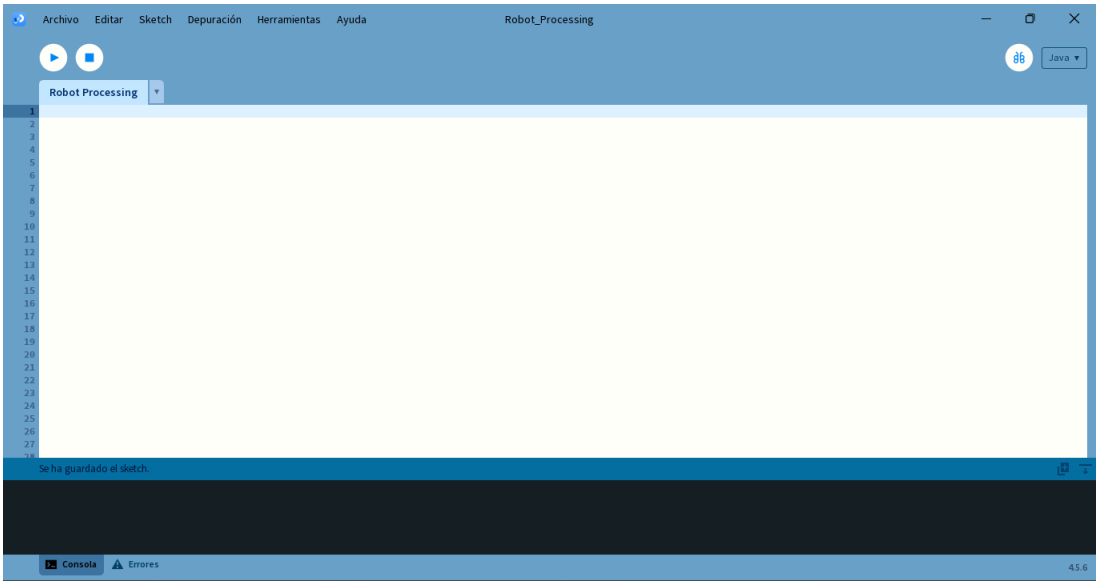




Processing lo encontraremos en la siguiente dirección:

💻Este equipo -> 💻Disco local -> 📁Archivos de programa -> 📁Processing -> Processing

Abrimos y creamos un nuevo **Sketch**, que es el espacio donde se escribirá y ejecutará el código. Para esta práctica el proyecto se guardó con el nombre: **Robot_Processing**



2. Creación y configuración del lienzo

   Para crear el área donde se realizará el dibujo se utiliza la función `size()` dentro de `setup()`.

```
void setup() {
      size(400, 400);
}
```

   En este caso se crea un lienzo de 400 píxeles de ancho por 400 píxeles de alto.

   Para establecer el color de fondo se utiliza `background()`:
   `background(25, 35, 45);`

   Los valores corresponden al sistema de color RGB.

   La estructura básica utilizada en el proyecto es:

```
void setup() {
  size(400, 400);
}

void draw() {
  background(25, 35, 45);
}
```

   `setup()` se ejecuta al iniciar el programa y `draw()` contiene las instrucciones de dibujo que se ejecutan mientras el programa está funcionando.

3. Primitivas geométricas

   Processing permite crear gráficos utilizando diferentes primitivas geométricas. Para construir el robot se utilizaron principalmente:

   Rectángulos

   `rect(x, y, ancho, alto);`

   Se utilizaron para formar la cabeza, el cuerpo, la pantalla, el panel y la sombra detrás de los ojos del robot.

   Círculos

   `circle(x, y, diámetro);`

   Se utilizaron para los ojos, la punta de la antena y el botón del panel.

   Líneas

   `line(x1, y1, x2, y2);`

   Se utilizaron para formar la antena y los brazos.

   Triángulos

   `triangle(x1, y1, x2, y2, x3, y3);`

   Se utilizaron para formar las manos del robot.

   Las coordenadas permiten establecer la posición de cada figura dentro del lienzo.

4. Colores, estilos y propiedades

   Para modificar la apariencia de las figuras se utilizaron diferentes funciones.

   Color de relleno: fill(46, 204, 113);

   `fill()` establece el color del interior de una figura.

   Color del contorno: stroke(46, 204, 113);

   `stroke()` permite establecer el color del contorno.

   Grosor del contorno: strokeWeight(4);

   `strokeWeight()` modifica el grosor del contorno o de una línea.

   Eliminar contorno: noStroke();

   `noStroke()` permite dibujar las siguientes figuras sin contorno.

   También se utilizó:

   rectMode(CENTER);
   ellipseMode(CENTER);

   para utilizar el centro de las figuras como referencia de su posición.

   Las propiedades de las figuras pueden modificarse directamente desde sus parámetros. Por ejemplo:
   circle(200, 265, 35);

   El primer y segundo valor representan la posición y el tercero el diámetro. Si se cambia `35` por `50`, el círculo aumenta de tamaño.

   De la misma manera, modificar los valores de posición permite mover la figura y cambiar `strokeWeight()` permite modificar el grosor del contorno.

5. Elaboración del robot

   Para realizar el robot se combinaron las primitivas y propiedades explicadas anteriormente.

   El diseño se dividió en diferentes partes:

   - Antena.
   - Cabeza.
   - Ojos.
   - Brazos.
   - Cuerpo.
   - Panel frontal.
   - Botón.
   - Manos.

   El código completo utilizado es:

```
void setup() {
  size(400, 400);
}

void draw() {
  background(25, 35, 45);

  rectMode(CENTER);
  ellipseMode(CENTER);

  // Antena
  stroke(46, 204, 113);
  strokeWeight(3);
  line(200, 110, 200, 75);
  line(200, 75, 215, 65);

  // Brazos
  stroke(30, 132, 73);
  strokeWeight(4);
  line(130, 250, 95, 290);
```

```
line(270, 250, 305, 290);

noStroke();

// Punta de la antena
fill(46, 204, 113);
circle(215, 65, 12);

// Cabeza
fill(46, 204, 113);
rect(200, 140, 120, 80, 15);

// Pantalla de la cara
fill(15, 30, 20);
rect(200, 135, 90, 35, 8);

// Ojos
fill(235, 247, 240);
circle(180, 135, 16);
circle(220, 135, 16);

// Cuello
fill(27, 94, 32);
rect(200, 185, 30, 10);

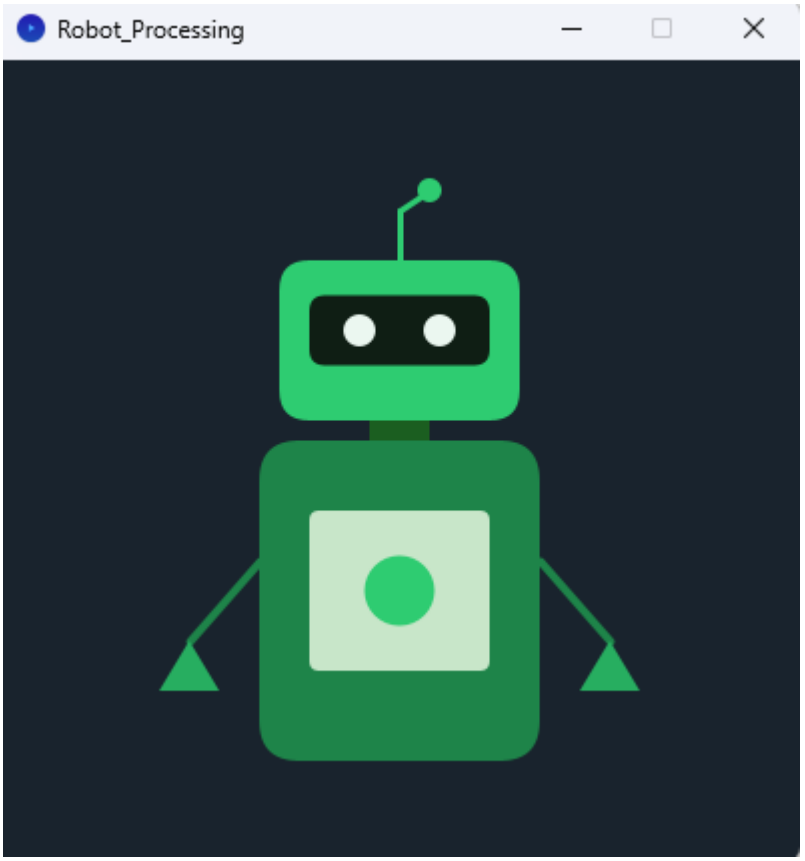
// Cuerpo
fill(30, 132, 73);
rect(200, 270, 140, 160, 20);

// Panel del cuerpo
fill(200, 230, 201);
rect(200, 265, 90, 80, 5);

// Botón
fill(46, 204, 113);
circle(200, 265, 35);
```

```
  // Manos
  fill(39, 174, 96);
  triangle(95, 290, 80, 315, 110, 315);
  triangle(305, 290, 290, 315, 320, 315);
}
```

El resultado obtenido es un robot construido completamente mediante código y primitivas geométricas.



6. Comparación con P5.js

Acción	Processing	P5.js
Crear lienzo	`size(400, 400);`	`createCanvas(400, 400);`
Función `setup`	`void setup()`	`function setup()`
Función `draw`	`void draw()`	`function draw()`
Relleno	`fill()`	`fill()`
Contorno	`stroke()`	`stroke()`
Grosor	`strokeWeight()`	`strokeWeight()`
Rectángulo	`rect()`	`rect()`
Círculo	`circle()`	`circle()`
Línea	`line()`	`line()`
Triángulo	`triangle()`	`triangle()`

Una de las principales diferencias observadas es la creación del lienzo. Processing utiliza `size()`, mientras que P5.js utiliza `createCanvas()`.

También cambia la sintaxis de las funciones principales, ya que Processing utiliza una sintaxis basada en Java, mientras que P5.js utiliza JavaScript.

A pesar de estas diferencias, las funciones relacionadas con el dibujo son muy similares, por lo que el robot pudo reproducirse utilizando instrucciones equivalentes.

7. Evidencia de código fuente y Resultado obtenido

Robot Processing

```
void setup() {
  size(400, 400);
}

void draw() {
  background(25, 35, 45);
  rectMode(CENTER);
  ellipseMode(CENTER);

  // Antena
  stroke(46, 204, 113);
  strokeWeight(3);
  line(200, 110, 200, 75);
  line(200, 75, 215, 65);

  // Brazos
  stroke(30, 132, 73);
  strokeWeight(4);
  line(130, 250, 95, 290);
  line(270, 250, 305, 290);
  noStroke();

  // Punta de la antena
  fill(46, 204, 113);
  circle(215, 65, 12);

  // Cabeza
  fill(46, 204, 113);
  rect(200, 140, 120, 80, 15);
```

```
  // Pantalla de la cara
  fill(15, 30, 20);
  rect(200, 135, 90, 35, 8);

  // Ojos
  fill(235, 247, 240);
  circle(180, 135, 16);
  circle(220, 135, 16);

  // Cuello
  fill(27, 94, 32);
  rect(200, 185, 30, 10);

  // Cuerpo
  fill(30, 132, 73);
  rect(200, 270, 140, 160, 20);

  // Panel del cuerpo
  fill(200, 230, 201);
  rect(200, 265, 90, 80, 5);

  // Botón
  fill(46, 204, 113);
  circle(200, 265, 35);

  // Manos
  fill(39, 174, 96);
  triangle(95, 290, 80, 315, 110, 315);
  triangle(305, 290, 290, 315, 320, 315);
}
```

